

A Practical Guide to Computational Mechanics

Finding Hidden Structure in Random Processes

John S Azariah
University of Technology, Sydney (UTS)
john.azariah@student.uts.edu.au

January 2026 — Draft

Abstract

This tutorial provides a comprehensive introduction to Computational Mechanics, the mathematical framework for understanding structure and complexity in stochastic processes. Starting from first principles, we build up to the six fundamental theorems that establish the ϵ -machine as the unique optimal predictor. We develop both the frequentist construction (via the CSSR algorithm) and the Bayesian perspective (via Dirichlet-multinomial inference). Throughout, we emphasize intuition and worked examples using the `emic` Python library.

Contents

I	Foundations	5
1	The Problem: Hidden Patterns in Randomness	5
1.1	Why Does This Matter?	5
2	Probability Refresher	6
2.1	Random Variables and Distributions	6
2.2	Conditional Probability	6
2.3	Independence	6
3	Information Theory Essentials	6
3.1	Entropy: Quantifying Uncertainty	6
3.2	Joint and Conditional Entropy	7
3.3	Mutual Information	7
4	Stochastic Processes	7
4.1	The Entropy Rate	8
4.2	Excess Entropy	8
II	Causal States and the ϵ-Machine	8
5	The Key Insight: Prediction-Equivalent Histories	8
6	Causal States: The Formal Definition	9
6.1	Morphs: The Predictive Signature	9

7	The ε-Machine	9
7.1	Unifilarity: A Key Property	10
III	The Six Theorems	10
8	Theorem 1: Prescience (Causal States Predict Optimally)	10
8.1	Background: The Prediction Problem	10
8.2	Notation and Setup	10
8.3	The Theorem	11
8.4	Proof Walkthrough	11
9	Theorem 2: Minimality (Simplest Prescient Representation)	12
9.1	The Compression Problem	12
9.2	Notation and Setup	12
9.3	The Theorem	12
9.4	Proof Walkthrough	12
10	Theorem 3: Uniqueness	13
10.1	The Identification Problem	13
10.2	Notation and Setup	13
10.3	The Theorem	13
10.4	Proof Walkthrough	13
11	Theorem 4: Minimal Stochasticity	14
11.1	The Transition Entropy Problem	14
11.2	Notation and Setup	14
11.3	The Theorem	14
11.4	Proof Walkthrough	15
12	Theorem 5: The Complexity Bound	15
12.1	Two Measures of Memory	15
12.2	Notation and Setup	15
12.3	The Theorem	16
12.4	Proof Walkthrough	16
13	Theorem 6: The Control Theorem	17
13.1	Memory and Predictability	17
13.2	Notation and Setup	17
13.3	The Theorem	17
13.4	Proof Walkthrough	18
13.5	Ashby's Law of Requisite Variety	18
13.6	Bringing It Together	19
14	Summary: What the Theorems Tell Us	19
IV	Complexity Measures	19
15	Statistical Complexity: C_μ	19
16	Entropy Rate: h_μ	20
17	The Complexity-Entropy Plane	20

V	Inference: From Data to ε-Machine	21
18	The CSSR Algorithm	21
18.1	The Idea	21
18.2	Homogeneity Testing	21
18.3	The Over-Splitting Problem	21
19	Spectral Learning: A Linear Algebra Approach	22
19.1	The Key Insight: Hankel Matrices	22
19.2	The Rank Reveals the States	22
19.3	The Algorithm	23
19.4	Why Spectral Methods Avoid Over-Splitting	23
19.5	Using emic's Spectral Learning	23
19.6	Algorithm Comparison	24
19.7	Using emic	24
20	Robustness to Observation Noise	25
21	The Bayesian Perspective	26
21.1	The Bayesian Setup	26
21.2	Dirichlet-Multinomial Framework	26
21.3	Gibbs Sampling for State Assignments	26
21.4	Advantages of BSI	26
21.5	Using emic's BSI	27
22	Causal State Merging (CSM)	27
22.1	Algorithm Intuition	27
22.2	Distance Metrics	27
22.3	Using emic's CSM	28
22.4	Trade-offs	28
23	Neural State Discovery (NSD)	28
23.1	Algorithm Intuition	28
23.2	Using emic's NSD	28
23.3	Trade-offs	29
VI	Worked Examples	29
24	Example 1: The Biased Coin	29
25	Example 2: The Golden Mean Process	29
26	Example 3: The Even Process	30
27	Example 4: Periodic Processes	30
VII	Going Further	31
28	Connections to Other Frameworks	31
28.1	Hidden Markov Models	31
28.2	Minimum Description Length	31
28.3	Kolmogorov Complexity	31

29 Open Questions	31
30 Further Reading	31

Part I

Foundations

1 The Problem: Hidden Patterns in Randomness

Key Idea

Even random-looking sequences often contain hidden structure. Computational Mechanics provides a principled way to discover this structure and quantify its complexity.

Consider flipping a fair coin repeatedly. The sequence looks like:

0 1 1 0 1 0 0 1 1 1 0 1 0 0 1 0 1 1 0 0 ...

Now consider a different rule: “After a 0, you *must* output a 1. After a 1, flip fairly.” This generates sequences like:

1 0 1 1 1 0 1 0 1 1 0 1 1 1 0 1 0 1 1 0 ...

Both sequences look random to the naked eye. But the second has *structure*—it was generated by a two-state machine that “remembers” whether the last symbol was 0 or 1.

The Central Question: *Given only the observed data, can we discover this hidden structure? And can we do so optimally—finding the simplest model that captures all the predictive information?*

Computational Mechanics answers both questions affirmatively. It provides:

1. A **definition** of what “structure” means (causal states)
2. A **construction** of the minimal predictive model (ϵ -machine)
3. **Proofs** that this model is optimal and unique
4. **Algorithms** to infer it from data
5. **Measures** to quantify the structure (C_μ, h_μ, E)

1.1 Why Does This Matter?

Structure detection appears across science:

- **Physics:** Finding order parameters in phase transitions
- **Neuroscience:** Discovering neural codes in spike trains
- **Finance:** Detecting regimes in market behavior
- **Biology:** Understanding gene regulatory dynamics
- **Language:** Modeling statistical structure in text

In each case, we observe a stream of symbols and want to infer the hidden process that generated them.

2 Probability Refresher

Before diving into Computational Mechanics, let's establish the probability concepts we'll need.

2.1 Random Variables and Distributions

A **random variable** X assigns outcomes to events. A **probability distribution** $P(X = x)$ tells us how likely each outcome is.

Biased Coin

A coin lands heads with probability $\theta = 0.7$:

$$P(X = \text{heads}) = 0.7$$

$$P(X = \text{tails}) = 0.3$$

Key rule: All probabilities sum to 1.

2.2 Conditional Probability

The **conditional probability** $P(X|Y)$ is the probability of X *given that we know* Y :

$$P(X = x|Y = y) = \frac{P(X = x, Y = y)}{P(Y = y)} \quad (1)$$

Intuition

Conditioning is like filtering. If we know $Y = y$, we restrict our attention to outcomes where $Y = y$ occurred.

2.3 Independence

Two random variables are **independent** if knowing one tells you nothing about the other:

$$P(X, Y) = P(X) \cdot P(Y) \quad \Leftrightarrow \quad P(X|Y) = P(X) \quad (2)$$

3 Information Theory Essentials

Information theory, developed by Claude Shannon in 1948 [15], provides the mathematical language for quantifying uncertainty and information. For a comprehensive treatment, see Cover and Thomas [4].

3.1 Entropy: Quantifying Uncertainty

Definition 3.1 (Shannon Entropy). The **entropy** of a random variable X is:

$$H[X] = - \sum_x P(x) \log_2 P(x) \quad (3)$$

measured in **bits**.

Intuition

Entropy measures *surprise*. If an event has probability p , its surprise is $-\log_2 p$ bits. Entropy is the *average* surprise.

Entropy of a Coin

Fair coin ($p = 0.5$):

$$H = -0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1 \text{ bit}$$

Biased coin ($p = 0.9$):

$$H = -0.9 \log_2 0.9 - 0.1 \log_2 0.1 \approx 0.47 \text{ bits}$$

Maximum entropy = maximum uncertainty = fair coin.

3.2 Joint and Conditional Entropy

Joint entropy measures uncertainty about both X and Y :

$$H[X, Y] = - \sum_{x,y} P(x, y) \log_2 P(x, y) \quad (4)$$

Conditional entropy measures remaining uncertainty about X after learning Y :

$$H[X|Y] = H[X, Y] - H[Y] \quad (5)$$

Key Idea

Conditioning reduces entropy (or keeps it the same): $H[X|Y] \leq H[X]$.

3.3 Mutual Information

Mutual information measures how much knowing Y tells us about X :

$$I(X; Y) = H[X] - H[X|Y] = H[Y] - H[Y|X] \quad (6)$$

Intuition

$I(X; Y)$ is the reduction in uncertainty about X from learning Y . It's symmetric: X tells us as much about Y as Y tells us about X .

4 Stochastic Processes

Definition 4.1 (Stochastic Process). A **stochastic process** is a sequence of random variables $\dots, X_{-1}, X_0, X_1, X_2, \dots$ taking values in an alphabet $\mathcal{A}$.

We denote:

- $\overleftarrow{S} = \dots X_{-2} X_{-1}$: the semi-infinite **past**
- $\overrightarrow{S} = X_0 X_1 X_2 \dots$: the semi-infinite **future**
- $X_0^{L-1} = X_0 X_1 \dots X_{L-1}$: a block of length L

Definition 4.2 (Stationarity). A process is **stationary** if its statistics don't change over time:

$$P(X_t^{t+L} = s) = P(X_0^L = s) \quad \text{for all } t, L$$

4.1 The Entropy Rate

For a stationary process, the **entropy rate** is the asymptotic per-symbol entropy:

$$h_\mu = \lim_{L \rightarrow \infty} \frac{1}{L} H[X_0^{L-1}] = \lim_{L \rightarrow \infty} H[X_L | X_0^{L-1}] \quad (7)$$

Intuition

h_μ measures the *irreducible randomness* per symbol—the uncertainty that remains even after seeing arbitrarily long histories.

4.2 Excess Entropy

The **excess entropy** [8] is the mutual information between past and future:

$$E = I(\overleftarrow{S}; \overrightarrow{S}) \quad (8)$$

Intuition

E measures how much the past tells us about the future—the “apparent memory” visible in the data.

Part II

Causal States and the ε -Machine

The concepts in this section were developed by Crutchfield and Young [6] and formalized fully by Shalizi and Crutchfield [13].

5 The Key Insight: Prediction-Equivalent Histories

Here’s the fundamental insight of Computational Mechanics:

Key Idea

Two histories are “the same” if they make the same predictions about the future.

Consider predicting the next symbol of a process. The full history $\overleftarrow{s} = \dots x_{-3}x_{-2}x_{-1}$ determines the conditional distribution $P(\overrightarrow{S} | \overleftarrow{S} = \overleftarrow{s})$.

But do we really need the *entire* history? Often, different histories lead to the same predictions.

Golden Mean Process

In the Golden Mean process, the histories “...1101” and “...0101” both end in 1. Since the rule is “after 1, flip fairly,” both histories predict:

$$P(\text{next} = 0) = 0.5, \quad P(\text{next} = 1) = 0.5$$

They’re *prediction-equivalent*—we can group them together!

6 Causal States: The Formal Definition

Definition 6.1 (Causal Equivalence). Two histories $\overleftarrow{s}, \overleftarrow{s}'$ are **causally equivalent**, written $\overleftarrow{s} \sim_\epsilon \overleftarrow{s}'$, if they have the same conditional distribution over all futures:

$$P(\overrightarrow{S} | \overleftarrow{S} = \overleftarrow{s}) = P(\overrightarrow{S} | \overleftarrow{S} = \overleftarrow{s}') \quad (9)$$

This is clearly an equivalence relation (reflexive, symmetric, transitive).

Definition 6.2 (Causal States). The **causal states** $\mathcal{S}$ are the equivalence classes under $\sim_\epsilon$:

$$\mathcal{S} = \{\text{histories}\} / \sim_\epsilon \quad (10)$$

The function $\epsilon : \{\text{histories}\} \rightarrow \mathcal{S}$ maps each history to its causal state.

Intuition

A causal state is a “sufficient summary” of the past. It retains exactly the information needed for optimal prediction—no more, no less.

6.1 Morphs: The Predictive Signature

Each causal state $\sigma \in \mathcal{S}$ has an associated **morph**—the conditional distribution over futures:

$$P(\overrightarrow{S} | \epsilon(\overleftarrow{s}) = \sigma) = P(\overrightarrow{S} | \overleftarrow{S} = \overleftarrow{s}) \quad \text{for any } \overleftarrow{s} \in \sigma \quad (11)$$

Different causal states have different morphs. The morph is what makes each state “who it is.”

7 The ϵ -Machine

Definition 7.1 (ϵ -Machine). The **ϵ -machine** of a process is the tuple $\{\mathcal{S}, \mathcal{A}, T, \pi\}$ where:

- $\mathcal{S}$ = the causal states
- $\mathcal{A}$ = the alphabet
- $T_{\sigma\sigma'}^{(x)}$ = probability of transitioning from σ to σ' while emitting symbol x
- π = the stationary distribution over states

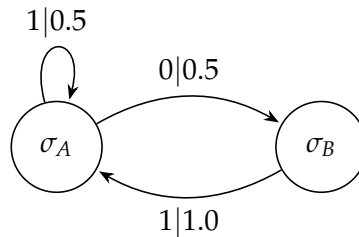


Figure 1: The ϵ -machine for the Golden Mean process. Edge labels show “symbol|probability.”

7.1 Unifilarity: A Key Property

Definition 7.2 (Unifilarity). An ε -machine is **unifilar** if each state-symbol pair determines a unique next state. That is, if we’re in state σ and emit symbol x , we know exactly which state comes next.

Lemma 7.1. *The ε -machine is always unifilar.*

Proof Sketch. If history $\overleftarrow{s}$ leads to state σ , then history $\overleftarrow{s}x$ leads to a specific state σ' that depends only on σ and x , not on the specific history $\overleftarrow{s}$. $\square$

Key Idea

Unifilarity means we can track the current state by observing the output sequence. No hidden randomness in the state transitions—all randomness is in choosing *which symbol* to emit.

Part III

The Six Theorems

The power of Computational Mechanics comes from six theorems established by Shalizi and Crutchfield [13] that together prove the ε -machine is the unique optimal predictor. These theorems are not merely mathematical curiosities—they provide the theoretical foundation for why ε -machines are the “right” way to represent process structure.

8 Theorem 1: Prescience (Causal States Predict Optimally)

8.1 Background: The Prediction Problem

Given a stochastic process $\{X_t\}$, suppose we’ve observed a history $\overleftarrow{s} = \dots x_{-2}x_{-1}x_0$ and want to predict the future $\overrightarrow{s} = x_1x_2\dots$. The fundamental question: *what should we remember about the past to make optimal predictions?*

Clearly, knowing the entire history $\overleftarrow{s}$ gives the best possible prediction. But histories are infinite—we can’t store them all. We need a *finite representation* that captures “what matters” for prediction.

8.2 Notation and Setup

Key symbols.

- $H[\cdot]$ — Shannon entropy [15], measuring uncertainty in bits
- $H[Y|X]$ — conditional entropy: average uncertainty in Y given knowledge of X
- $\overrightarrow{s}^L = X_1X_2\dots X_L$ — the next L symbols of the future
- $R : \overleftarrow{S} \rightarrow \mathcal{R}$ — any function that maps histories to a finite set of “states”
- $\mathcal{S}$ — the set of causal states (our special partition based on predictive equivalence)

What is a “partition”? A partition R groups histories into equivalence classes. Given history $\overleftarrow{s}$, we assign it to state $R(\overleftarrow{s})$. Different histories may map to the same state. The key question: which partition gives the best predictions?

8.3 The Theorem

Theorem 1: Prescience [13]

Theorem 8.1 (Causal States are Maximally Prescient). *For any partition of histories R , and any prediction horizon L :*

$$H[\vec{S}^L | R] \geq H[\vec{S}^L | \mathcal{S}] \quad (12)$$

where $\mathcal{S}$ denotes the causal state partition.

Reading the inequality.

- Left side: “uncertainty about the next L symbols, given that we know which class R assigns to our history”
- Right side: “uncertainty about the next L symbols, given that we know the causal state”
- The inequality: causal states leave us with *less or equal* uncertainty—they never predict worse than any alternative

8.4 Proof Walkthrough

The proof proceeds in three steps:

Step 1: The full history is optimal. By the data processing inequality [4], conditioning on more information never increases entropy:

$$H[\vec{S}^L | \overleftarrow{S}] \leq H[\vec{S}^L | R(\overleftarrow{S})] \quad (13)$$

for any function R . The full history $\overleftarrow{S}$ gives the best possible prediction.

Step 2: Causal states preserve predictive information. By the *definition* of causal states (Definition 6.2), two histories $\overleftarrow{s}$ and $\overleftarrow{s}'$ are in the same causal state if and only if they have identical conditional distributions over futures:

$$P(\vec{S} | \overleftarrow{s}) = P(\vec{S} | \overleftarrow{s}') \quad (14)$$

This means knowing the causal state gives exactly the same predictive distribution as knowing the full history:

$$H[\vec{S}^L | \mathcal{S}] = H[\vec{S}^L | \overleftarrow{S}] \quad (15)$$

Step 3: Any other partition is at least as uncertain. Any other partition R either:

- Is a *refinement* of $\mathcal{S}$ (distinguishes histories that $\mathcal{S}$ groups together), in which case it matches $\mathcal{S}$ ’s predictions, or
- Is *coarser* than $\mathcal{S}$ (groups histories that $\mathcal{S}$ distinguishes), in which case it loses predictive information

In either case, $H[\vec{S}^L | R] \geq H[\vec{S}^L | \mathcal{S}]$. □

Intuition

No matter how you group histories, you can’t predict the future better than by using causal states. The causal states are *sufficient statistics* for prediction—they capture everything predictively relevant about the past.

Corollary 8.2 (Sufficient Statistics). *The causal states $\mathcal{S}$ are sufficient statistics for predicting $\vec{S}$. In the language of statistics, $\overleftarrow{S} \rightarrow \mathcal{S} \rightarrow \vec{S}$ forms a Markov chain.*

9 Theorem 2: Minimality (Simplest Prescient Representation)

9.1 The Compression Problem

Theorem 8.1 tells us causal states predict optimally—but are they the *simplest* way to achieve optimal prediction? Perhaps there’s a more compact representation that predicts just as well?

This is where Theorem 2 comes in: it shows that among all optimal predictors, causal states use the *least memory*.

9.2 Notation and Setup

Key symbols.

- $\hat{R}$ — any alternative representation (partition) of histories
- $H[R]$ — the entropy of the state distribution: $H[R] = -\sum_r P(R = r) \log_2 P(R = r)$
- $C_\mu \equiv H[S]$ — statistical complexity [6], the entropy of the causal state distribution
- “ $\hat{R}$ is prescient” — means $H[\vec{S}^L | \hat{R}] = H[\vec{S}^L | S]$ for all L

What does $H[R]$ measure? The entropy $H[R]$ quantifies the “size” of representation R in an information-theoretic sense. It measures how many bits we need on average to specify which state we’re in. A representation with many equally-likely states has high entropy; one dominated by a single state has low entropy.

9.3 The Theorem

Theorem 2: Minimality [13]

Theorem 9.1 (Causal States are Minimal). *Among all representations $\hat{R}$ that are as prescient as causal states:*

$$H[\hat{R}] \geq H[S] = C_\mu \quad (16)$$

Reading the inequality. If $\hat{R}$ predicts as well as causal states, then $\hat{R}$ must use at least as much memory (measured by entropy) as causal states. You cannot achieve optimal prediction with a simpler representation.

9.4 Proof Walkthrough

The proof uses a fundamental result from information theory.

Step 1: Set up the information flow. Consider the chain $\overleftarrow{S} \rightarrow \hat{R} \rightarrow \overrightarrow{S}$. If $\hat{R}$ is prescient (predicts as well as knowing the full past), then by the data processing inequality [4]:

$$I[\overleftarrow{S}; \overrightarrow{S}] = I[\hat{R}; \overrightarrow{S}] \quad (17)$$

where I denotes mutual information. The representation $\hat{R}$ captures *all* the predictive information in the past.

Step 2: Apply the information-theoretic identity. The mutual information satisfies:

$$I[\hat{R}; \overrightarrow{S}] \leq H[\hat{R}] \quad (18)$$

with equality only when $\hat{R}$ is a deterministic function of $\overrightarrow{S}$ (which is generally not the case).

Step 3: Causal states achieve the bound. Causal states are defined as the *minimal* sufficient statistics for prediction. They achieve:

$$H[\mathcal{S}] = C_\mu = \min_{\hat{R} \text{ prescient}} H[\hat{R}] \quad (19)$$

Any other prescient $\hat{R}$ must either match this entropy or exceed it. $\square$

Intuition

If you want optimal prediction, you can't do it with a simpler representation than causal states. The ε -machine has the smallest “memory footprint” among all optimal predictors.

Why this matters. This is where *Occam's Razor becomes a theorem!* We often invoke parsimony as a heuristic—prefer simpler models. Theorem 2 proves that for prediction, the simplest optimal model is unique and well-defined: it's the ε -machine.

10 Theorem 3: Uniqueness

10.1 The Identification Problem

We've established that causal states are:

1. **Prescient:** They predict as well as knowing the full history (Theorem 8.1)
2. **Minimal:** No simpler representation predicts as well (Theorem 9.1)

But could there be *multiple* representations that are both optimal and minimal? Perhaps different constructions lead to equally good but structurally different models?

Theorem 3 answers decisively: *no*. The ε -machine is unique.

10.2 Notation and Setup

Key concepts.

- “ $\hat{R}$ is prescient” — $\hat{R}$ predicts as well as causal states for all horizons
- “ $\hat{R}$ is minimal” — $H[\hat{R}] = C_\mu$ (achieves the lower bound from Theorem 9.1)
- “Equivalent up to relabeling” — there exists a bijection $\phi : \hat{\mathcal{R}} \rightarrow \mathcal{S}$ such that $\phi(\hat{R}(\overleftarrow{s})) = \mathcal{S}(\overleftarrow{s})$ for all histories $\overleftarrow{s}$

10.3 The Theorem

Theorem 3: Uniqueness [13]

Theorem 10.1 (Causal States are Unique). *If $\hat{R}$ is prescient and $H[\hat{R}] = C_\mu$, then $\hat{R}$ is equivalent to $\mathcal{S}$ up to relabeling.*

10.4 Proof Walkthrough

The proof shows that any optimal-minimal representation must be isomorphic to causal states.

Step 1: Prescience implies refinement. If $\hat{R}$ is prescient, it must distinguish histories that have different predictive futures. Since causal states are defined by predictive equivalence, $\hat{R}$ must be at least as fine as $\mathcal{S}$ —it cannot merge histories that $\mathcal{S}$ separates.

Formally: if $\mathcal{S}(\overleftarrow{s}) \neq \mathcal{S}(\overleftarrow{s}')$, then $\hat{R}(\overleftarrow{s}) \neq \hat{R}(\overleftarrow{s}')$.

Step 2: Minimality implies no finer than $\mathcal{S}$. If $\hat{\mathcal{R}}$ is finer than $\mathcal{S}$ (distinguishes histories that $\mathcal{S}$ groups), then $H[\hat{\mathcal{R}}] > H[\mathcal{S}] = C_\mu$. But we assumed $H[\hat{\mathcal{R}}] = C_\mu$.

Contradiction. So $\hat{\mathcal{R}}$ cannot be finer than $\mathcal{S}$.

Step 3: Combine to get equality. From Steps 1 and 2: $\hat{\mathcal{R}}$ is at least as fine as $\mathcal{S}$, but not finer. Therefore $\hat{\mathcal{R}}$ and $\mathcal{S}$ define the same partition, possibly with different state labels. $\square$

Intuition

There's only one way to be both optimal and minimal. The ε -machine is not just *an* optimal predictor—it's *the* optimal predictor. Any other optimal-minimal model is the ε -machine in disguise.

The “canonical model” property. Uniqueness is crucial for interpretation. When we infer an ε -machine from data, we're not finding one of many possible models—we're recovering *the* intrinsic structure of the process. This justifies calling the ε -machine the “canonical” representation.

11 Theorem 4: Minimal Stochasticity

11.1 The Transition Entropy Problem

Theorems 1–3 establish that causal states are the unique optimal-minimal representation. But there's another property we might desire: *predictable dynamics*.

Given that we're in state σ , how uncertain are we about the *next* state σ' ? Ideally, we'd like the state evolution to be as deterministic as possible—randomness should come from the symbols, not from hidden state transitions.

11.2 Notation and Setup

Key symbols.

- $\mathcal{S}$ — the current causal state
- $\mathcal{S}'$ — the causal state at the next time step (after observing symbol X)
- $H[\mathcal{S}'|\mathcal{S}]$ — transition entropy: uncertainty about the next state given the current state
- $H[\mathcal{S}'|\mathcal{S}, X]$ — uncertainty about the next state given current state *and* the emitted symbol

Unifilarity revisited. Recall that ε -machines are *unifilar*: given the current state σ and the next symbol x , the next state σ' is determined uniquely. This means $H[\mathcal{S}'|\mathcal{S}, X] = 0$ —zero uncertainty once we know the symbol.

But $H[\mathcal{S}'|\mathcal{S}]$ is generally nonzero, because we don't know which symbol will be emitted.

11.3 The Theorem

Theorem 4: Minimal Stochasticity [13]

Theorem 11.1 (ε -Machines are Minimally Stochastic). *For all prescient representations $\hat{\mathcal{R}}$:*

$$H[\hat{\mathcal{R}}'|\hat{\mathcal{R}}] \geq H[\mathcal{S}'|\mathcal{S}] \quad (20)$$

where $\mathcal{S}'$ and $\hat{\mathcal{R}}'$ denote the next-step states.

Reading the inequality. The left side is the transition entropy for representation $\hat{R}$. The right side is the transition entropy for causal states. The inequality says: no prescient representation has more deterministic dynamics than the ε -machine.

11.4 Proof Walkthrough

Step 1: Decompose transition entropy. For any representation $\hat{R}$:

$$H[\hat{R}'|\hat{R}] = H[\hat{R}'|\hat{R}, X] + I[\hat{R}'; X|\hat{R}] \quad (21)$$

The first term is the “internal noise”—randomness in transitions not explained by the symbol. The second term is predictable from X .

Step 2: Unifilarity minimizes internal noise. For the ε -machine, unifilarity gives $H[S'|\mathcal{S}, X] = 0$. The internal noise vanishes completely.

For a non-unifilar representation, the same symbol might lead to different next states—so $H[\hat{R}'|\hat{R}, X] > 0$.

Step 3: Combine the bounds. Since $H[\hat{R}'|\hat{R}, X] \geq 0 = H[S'|\mathcal{S}, X]$, and prescient representations preserve the predictive information, we get the inequality. $\square$

Intuition

The transitions between causal states are as deterministic as possible. All randomness in the ε -machine comes from symbol selection, not from hidden state dynamics. There’s no “extra” noise hiding in the state transitions.

Connection to hidden Markov models. Standard HMMs can have stochastic hidden-to-hidden transitions (even given the observation). The ε -machine’s unifilarity property eliminates this source of uncertainty, making it a “maximally observable” HMM [12].

12 Theorem 5: The Complexity Bound

12.1 Two Measures of Memory

Theorems 1–4 characterize the ε -machine’s optimality properties. Theorems 5 and 6 relate the statistical complexity C_μ to other information-theoretic quantities, revealing deep connections.

Theorem 5 relates C_μ to the *excess entropy* E —a measure of “apparent memory” that can be estimated directly from time series data.

12.2 Notation and Setup

Key quantities.

- $E \equiv I[\overleftarrow{\mathcal{S}}; \overrightarrow{\mathcal{S}}]$ — the **excess entropy** [8], also called “effective measure complexity”
- $I[X; Y] = H[X] - H[X|Y]$ — mutual information between X and Y
- $C_\mu \equiv H[\mathcal{S}]$ — statistical complexity, the entropy of causal states
- $H[\mathcal{S} | \overrightarrow{\mathcal{S}}]$ — residual uncertainty about the current state even knowing the entire future

What does excess entropy measure? $E = I[\overleftarrow{S}; \overrightarrow{S}]$ quantifies how much the past tells you about the future, summed over all horizons. It measures the *total correlation* between past and future—the “amount of mutual prediction” possible.

Crucially, E can be estimated from data without knowing the ε -machine structure, via:

$$E = \lim_{L \rightarrow \infty} [H[X_1^L] - L \cdot h_\mu] \quad (22)$$

where $H[X_1^L]$ is the block entropy of L -symbol sequences.

What does C_μ measure? $C_\mu = H[S]$ is the entropy of the causal state distribution—the average number of bits needed to specify the current state. It measures the “true memory” that the process uses internally.

12.3 The Theorem

Theorem 5: The Bound [13]

Theorem 12.1 (Excess Entropy Bound). *The statistical complexity bounds the excess entropy:*

$$E \leq C_\mu \quad (23)$$

with equality if and only if $H[S | \overrightarrow{S}] = 0$.

Reading the inequality. The “apparent memory” (visible in correlations) is always less than or equal to the “true memory” (stored in causal states). Some structure may be hidden!

12.4 Proof Walkthrough

Step 1: Express E in terms of states. Since causal states are sufficient statistics for prediction:

$$E = I[\overleftarrow{S}; \overrightarrow{S}] = I[S; \overrightarrow{S}] \quad (24)$$

The past-future mutual information equals the state-future mutual information.

Step 2: Use the mutual information bound. By a standard identity:

$$I[S; \overrightarrow{S}] = H[S] - H[S | \overrightarrow{S}] \quad (25)$$

Since entropy is nonnegative:

$$I[S; \overrightarrow{S}] \leq H[S] = C_\mu \quad (26)$$

with equality iff $H[S | \overrightarrow{S}] = 0$.

Step 3: Interpret the gap. The quantity $\chi \equiv C_\mu - E = H[S | \overrightarrow{S}]$ is called the **crypticity** [11]. It measures how much of the process’s internal memory is *not* reflected in past-future correlations—“hidden” structure that doesn’t affect correlations but is still needed for optimal prediction. $\square$

Intuition

The “apparent memory” E (visible in correlations) is always less than or equal to the “true memory” C_μ (stored in causal states). The gap is *crypticity*—structure hidden from correlation analysis.

When $E < C_\mu$: Cryptic Processes

Consider a process that “remembers” information for a long time before using it. The current state affects the distant future but not the immediate future. Standard correlation measures (which focus on nearby time points) may miss this structure, giving $E < C_\mu$. A concrete example: a counter that increments silently for 1000 steps, then resets with a special symbol. The state (counter value) is highly structured (C_μ is large), but consecutive symbols are nearly independent (E is small).

Practical implication. When analyzing data, E gives a *lower bound* on the process’s true complexity C_μ . If you find $E = 2$ bits, you know the process has *at least* 2 bits of memory—possibly much more.

13 Theorem 6: The Control Theorem

13.1 Memory and Predictability

The final theorem relates the statistical complexity to *how much prediction is possible*—the gap between maximum entropy and the actual entropy rate.

This theorem has a beautiful interpretation in terms of *control theory*: how much memory is needed to exploit the structure in a process?

13.2 Notation and Setup

Key quantities.

- $H[X]$ — entropy of a single symbol: $H[X] = -\sum_x P(x) \log_2 P(x)$
- h_μ — entropy rate [4], the average surprise per symbol given infinite context:

$$h_\mu = \lim_{L \rightarrow \infty} H[X_L | X_1, \dots, X_{L-1}] \quad (27)$$

- $H[X] - h_\mu$ — the **redundancy**: how much each symbol is constrained by process structure
- C_μ — statistical complexity

Interpreting redundancy. $H[X]$ is the entropy if symbols were independent (maximum entropy for the given alphabet). h_μ is the actual per-symbol entropy accounting for correlations. The difference $H[X] - h_\mu$ is the “predictable part”—how much better than random guessing you can do.

13.3 The Theorem

Theorem 6: Control [13]

Theorem 13.1 (Control Theorem). *For the ε -machine:*

$$H[X] - h_\mu \leq C_\mu \quad (28)$$

where $H[X]$ is the single-symbol entropy.

Reading the inequality. The “exploitable structure” (how much better than random you can predict) is bounded by the memory cost C_μ . You can’t exploit more structure than you can afford to remember.

13.4 Proof Walkthrough

Step 1: Express h_μ in terms of states. Given the current causal state $\mathcal{S}$, the next symbol X is generated with known probabilities. Thus:

$$h_\mu = H[X|\mathcal{S}] \quad (29)$$

The entropy rate is the expected symbol entropy, averaged over states.

Step 2: Apply the entropy chain rule.

$$H[X] = I[X;\mathcal{S}] + H[X|\mathcal{S}] \quad (30)$$

$$= I[X;\mathcal{S}] + h_\mu \quad (31)$$

Rearranging: $H[X] - h_\mu = I[X;\mathcal{S}]$.

Step 3: Bound the mutual information. By the standard bound on mutual information:

$$I[X;\mathcal{S}] \leq H[\mathcal{S}] = C_\mu \quad (32)$$

Combining: $H[X] - h_\mu \leq C_\mu$. □

13.5 Ashby's Law of Requisite Variety

The Control Theorem has a beautiful connection to **Ashby's Law of Requisite Variety** [1], a foundational principle from cybernetics (the study of control and communication in animals and machines).

Key Idea

Law of Requisite Variety (Ashby, 1956): A controller can only control a system to the extent that it has as much “variety” (possible states) as the disturbances it must counteract.

The intuition. Imagine you're playing a game against an adversary. Your opponent can make N different moves. To guarantee you can respond appropriately to each move, you need at least N possible responses. If you have fewer options than your opponent, there will be some situations you simply cannot handle.

Application to prediction. In our context:

- The “disturbances” are the unpredictable aspects of the process—measured by h_μ
- The “variety needed to control” is measured by C_μ —your memory capacity
- The “exploitable regularity” is $H[X] - h_\mu$ —how much you can predict beyond random guessing

The Control Theorem says: to exploit k bits of predictability, you need at least k bits of memory. You can't “get something for nothing”—prediction requires proportional memory investment.

Intuition

Ashby's Law is why prediction is hard! To predict a complex process, you need a complex model. The ε -machine achieves the theoretical minimum—it needs exactly C_μ bits to exploit all available structure.

Connection to rate-distortion theory. The Control Theorem is also analogous to results in rate-distortion theory [4]: there’s a fundamental trade-off between compression (low C_μ) and prediction quality (low h_μ). The ε -machine sits on the optimal frontier of this trade-off.

13.6 Bringing It Together

Theorems 5 and 6 give us two inequalities:

$$\begin{aligned} E &\leq C_\mu && \text{(excess entropy bound)} && (33) \\ H[X] - h_\mu &\leq C_\mu && \text{(control bound)} && (34) \end{aligned}$$

These show that C_μ is a *central quantity*—it bounds both the measurable correlations (E) and the exploitable predictability ($H[X] - h_\mu$). The statistical complexity truly captures the “computational cost” of the process.

14 Summary: What the Theorems Tell Us

Theorem	What It Says
1. Prescience	Causal states predict as well as the full history
2. Minimality	No simpler representation predicts as well
3. Uniqueness	The ε -machine is the <i>only</i> optimal-minimal model
4. Min. Stochasticity	State transitions are as deterministic as possible
5. Bound	Apparent memory $\leq$ true memory: $E \leq C_\mu$
6. Control	Memory bounds exploitable predictability

Together, these theorems justify calling the ε -machine the “canonical” representation of a process’s structure.

Part IV

Complexity Measures

15 Statistical Complexity: C_μ

Definition 15.1 (Statistical Complexity). The **statistical complexity** C_μ is the entropy of the causal state distribution:

$$C_\mu = H[\mathcal{S}] = - \sum_{\sigma \in \mathcal{S}} \pi(\sigma) \log_2 \pi(\sigma) \quad (35)$$

where $\pi(\sigma)$ is the stationary probability of state σ .

Intuition

C_μ measures the *memory* required for optimal prediction. It’s how many bits the process “remembers” about its past.

Computing C_μ for Golden Mean

The Golden Mean has two states with stationary distribution $\pi(A) = 2/3, \pi(B) = 1/3$:

$$\begin{aligned} C_\mu &= -\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3} \\ &= \frac{2}{3}(0.585) + \frac{1}{3}(1.585) \\ &\approx 0.918 \text{ bits} \end{aligned}$$

16 Entropy Rate: h_μ

Definition 16.1 (Entropy Rate from ε -Machine). For a unifilar ε -machine, the entropy rate is:

$$h_\mu = H[X|\mathcal{S}] = - \sum_{\sigma} \pi(\sigma) \sum_x P(x|\sigma) \log_2 P(x|\sigma) \quad (36)$$

Intuition

h_μ measures the *irreducible randomness* per symbol. It's the uncertainty that remains even knowing the current state.

Computing h_μ for Golden Mean

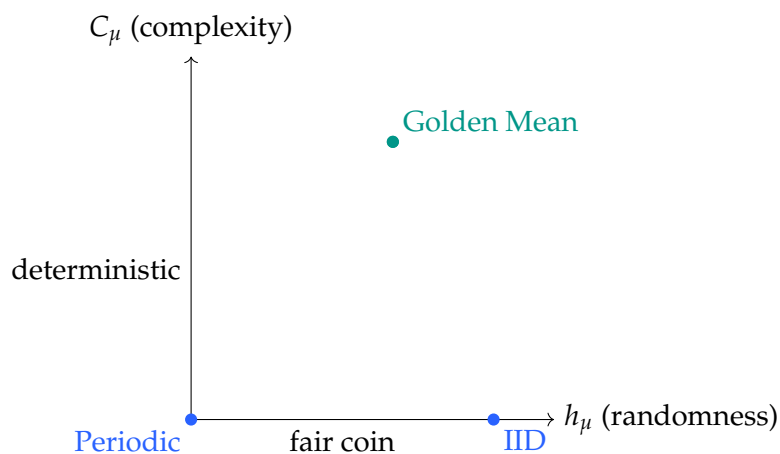
State A emits 0 or 1 with equal probability: $H[X|A] = 1$ bit.

State B always emits 1: $H[X|B] = 0$ bits.

$$\begin{aligned} h_\mu &= \pi(A) \cdot H[X|A] + \pi(B) \cdot H[X|B] \\ &= \frac{2}{3} \cdot 1 + \frac{1}{3} \cdot 0 \\ &= \frac{2}{3} \approx 0.667 \text{ bits} \end{aligned}$$

17 The Complexity-Entropy Plane

Different processes occupy different regions of the (h_μ, C_μ) plane:



- **Bottom-left** ($h_\mu = 0, C_\mu = 0$): Constant sequences (no randomness, no structure)
- **Bottom-right** ($h_\mu = 1, C_\mu = 0$): IID fair coin (maximum randomness, no structure)
- **Middle**: Interesting processes with both structure and randomness

The “edge of chaos” lives in the middle—complex enough to be interesting, structured enough to be predictable.

Part V

Inference: From Data to ε -Machine

18 The CSSR Algorithm

CSSR (Causal State Splitting Reconstruction) is the standard algorithm for inferring ε -machines from data [14].

18.1 The Idea

1. Start with a single state containing all histories
2. Test if histories in a state are “homogeneous” (same predictive distribution)
3. If not, **split** the state based on predictive differences
4. Iterate until all states are homogeneous

18.2 Homogeneity Testing

Two histories are in the same state only if their next-symbol distributions are statistically indistinguishable. CSSR uses a χ^2 test or similar.

18.3 The Over-Splitting Problem

CSSR’s Achilles Heel

CSSR can **over-split** states when data is limited. If two histories happen to have different empirical distributions due to sampling noise, CSSR may incorrectly split them into separate states—even when they belong to the same true causal state.

Why does over-splitting happen? Consider the Golden Mean process with its 2 true states. Suppose we observe:

- History “01”: followed by 0 in 48% of cases, 1 in 52%
- History “11”: followed by 0 in 52% of cases, 1 in 48%

The true distribution for both is 50/50, but sampling noise created a difference. If our significance threshold is too strict, CSSR’s χ^2 test rejects the hypothesis that they’re equal—and we get 3+ states instead of 2.

Consequences of over-splitting.

- **Inflated complexity**: C_μ is overestimated
- **Poor generalization**: The model overfits to training data
- **Spurious structure**: We “discover” patterns that don’t exist

CSSR's mitigation strategies. CSSR includes heuristics to reduce over-splitting:

1. **Significance level:** Higher α (e.g., 0.05 vs 0.001) makes tests more lenient
2. **State merging:** After splitting, attempt to merge states with similar distributions
3. **More data:** With enough samples, empirical distributions converge to truth

But these are heuristics, not guarantees. For a principled solution, we turn to spectral methods.

19 Spectral Learning: A Linear Algebra Approach

Spectral learning [10, 3] offers an elegant alternative to CSSR's hypothesis testing. Instead of iteratively splitting states, it recovers the model structure in one shot using linear algebra.

19.1 The Key Insight: Hankel Matrices

The central object is the **Hankel matrix** H , which organizes sequence probabilities:

$$H_{ij} = P(\text{prefix } p_i, \text{suffix } s_j) \quad (37)$$

where p_i ranges over history strings and s_j ranges over future strings.

A Small Hankel Matrix

For a binary alphabet with histories/futures up to length 2:

$$H = \begin{pmatrix} P(\epsilon, \epsilon) & P(\epsilon, 0) & P(\epsilon, 1) & P(\epsilon, 00) & \dots \\ P(0, \epsilon) & P(0, 0) & P(0, 1) & P(0, 00) & \dots \\ P(1, \epsilon) & P(1, 0) & P(1, 1) & P(1, 00) & \dots \\ P(00, \epsilon) & P(00, 0) & P(00, 1) & P(00, 00) & \dots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{pmatrix}$$

where ϵ is the empty string and $P(p, s) = P(\text{seeing } p \text{ then } s)$.

19.2 The Rank Reveals the States

Here's the magic: **the rank of H equals the number of causal states!**

Hankel Rank Theorem

For a stationary process generated by a minimal HMM with k states:

$$\text{rank}(H) = k \quad (38)$$

Why does this work? The Hankel matrix can be factored as:

$$H = \Omega \cdot D \cdot \Lambda^\top \quad (39)$$

where:

- Ω (history $\rightarrow$ state): maps histories to state distributions
- D : diagonal matrix of stationary state probabilities
- Λ (state $\rightarrow$ future): maps states to future distributions

Both Ω and Λ have rank k (the number of states), so their product has rank $\leq k$. For a minimal model, the rank is exactly k .

19.3 The Algorithm

Spectral learning proceeds as:

1. **Estimate H :** Count substring frequencies from data
2. **Compute SVD:** $H \approx U\Sigma V^\top$
3. **Determine rank:** Find the “elbow” where singular values drop off
4. **Extract model:** Recover transition matrices from the SVD factors

19.4 Why Spectral Methods Avoid Over-Splitting

Key Idea

Spectral methods determine the number of states **globally** from the rank of H , not **locally** from pairwise tests between histories.

The key advantages:

- **Noise averaging:** The Hankel matrix aggregates information from *all* prefix-suffix pairs. Noise in individual entries averages out.
- **Rank stability:** Small perturbations to H cause small perturbations to singular values. We can reliably identify large gaps in the spectrum.
- **No cascading errors:** CSSR’s splits are sequential—early mistakes propagate. Spectral methods make one global decision.
- **Principled regularization:** We can use the singular value spectrum to balance model complexity against fit.

Spectral vs CSSR on Noisy Data

Consider 10,000 samples from the Golden Mean (2 true states):

- **CSSR** (significance=0.01): Often infers 3–4 states
- **Spectral:** Singular values are $[\sigma_1, \sigma_2, \sigma_3, \dots] = [1.2, 0.8, 0.02, 0.01, \dots]$
Clear gap after 2 values $\Rightarrow$ correctly infers 2 states

19.5 Using emic’s Spectral Learning

```
from emic import Spectral, SpectralConfig
from emic.sources import GoldenMean

source = GoldenMean(p=0.5)
data = source.generate(10_000)  # Spectral is sample-efficient

# Default config uses adaptive parameters based on sample size
config = SpectralConfig()  # Automatically sets max_history,
    min_count
spectral = Spectral(config)
result = spectral.infer(data)

print(f"Inferred {result.machine.num_states} states")
```

```
# For expert control, parameters can be set explicitly:
# config = SpectralConfig(
#     max_history=10,          # Fixed history length (None = adaptive)
# )
#     rank_threshold=0.001,   # Singular value cutoff (lower = fewer
#                             # states)
#     rank=2,                 # Or fix rank directly when known
#     min_count=5             # Minimum observations per history (
#                             # None = adaptive)
# )
```

19.6 Algorithm Comparison

The following table summarizes the key trade-offs between all five inference algorithms. Based on our benchmarks across diverse processes, we provide practical guidance for algorithm selection.

Property	CSSR	Spectral	CSM	NSD	BSI
Sample efficiency	Moderate	High	Moderate	Moderate	Low
Accuracy ($N \geq 1K$)	80%	98%	45%	75%	25%
Over-splitting risk	High	Low	Low	Low	N/A
Under-splitting risk	Low	Moderate	High	Moderate	High
Interpretability	Direct	Post-process	Direct	Low	Probabilistic
Unifilarity guarantee	Yes	No	No	No	No
Max states control	No	Via rank	No	Yes	Yes

Intuition

Recommendation: Start with **Spectral** for most applications—it achieves 98% accuracy at $N \geq 1000$ samples with adaptive defaults. Use **CSSR** when you need guaranteed unifilarity. Use **NSD** or **BSI** when you need to bound the number of states. Use **CSM** when you want hierarchical insights into state structure.

19.7 Using emic

```
from emic import CSSR, CSSRConfig
from emic.sources import GoldenMean

# Generate data
source = GoldenMean(p=0.5)
data = source.generate(100_000)

# Configure and run CSSR
config = CSSRConfig(max_history=5, significance=0.01)
cssr = CSSR(config)
result = cssr.infer(data)

print(f"Inferred {result.machine.num_states} states")
```


20 Robustness to Observation Noise

Real-world data is noisy. How do inference algorithms behave when observations are corrupted? We tested all five algorithms on the Golden Mean process with bit-flip noise at levels $\varepsilon \in [0, 0.5]$ (50 repetitions each, $N = 10,000$ samples).

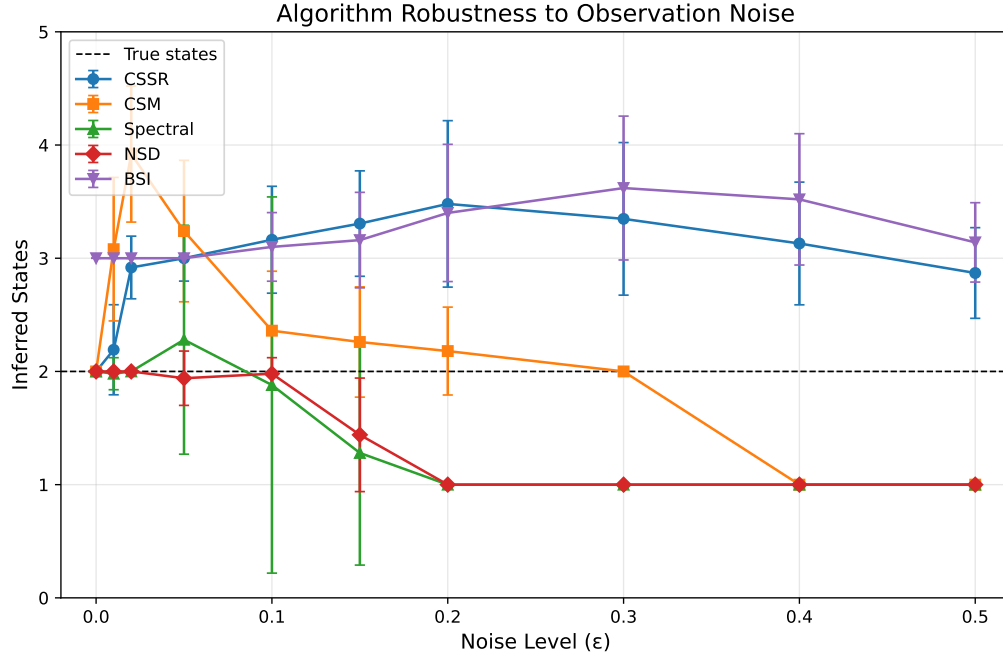


Figure 2: Inferred states vs. noise level for the Golden Mean process (true states = 2). Error bars show ± 1 standard deviation across 50 runs.

Algorithm	$\varepsilon = 0$	$\varepsilon = 0.05$	$\varepsilon = 0.1$	$\varepsilon = 0.2$	$\varepsilon = 0.5$
CSSR	2.0±0.0	3.0±0.2	3.2±0.5	3.5±0.7	2.9±0.4
CSM	2.0±0.0	3.2±0.6	2.4±0.5	2.2±0.4	1.0±0.0
Spectral	2.0±0.0	2.3±1.0	1.9±1.7	1.0±0.0	1.0±0.0
NSD	2.0±0.0	1.9±0.2	2.0±0.1	1.0±0.0	1.0±0.0
BSI	3.0±0.0	3.0±0.0	3.1±0.3	3.4±0.6	3.1±0.4

Intuition

Key findings:

- **CSSR** over-splits with noise, finding 3+ states at moderate noise levels
- **Spectral/NSD** under-split at high noise, collapsing to 1 state
- **CSM** shows bipolar behavior: over-splits at low noise, under-splits at high noise
- **BSI** consistently over-splits (even at $\varepsilon = 0$), suggesting prior/hyperparameter tuning is needed

No algorithm is universally robust. For noisy data, cross-validate with multiple algorithms and be skeptical of results.

21 The Bayesian Perspective

CSSR takes a frequentist approach: test hypotheses about state equality. An alternative is **Bayesian Structural Inference (BSI)**, which provides full posterior distributions over model structures.

The Bayesian approach to inference is well-established in machine learning [2]. For a general introduction to HMM inference, see Rabiner's tutorial [12].

21.1 The Bayesian Setup

Instead of testing, we compute:

$$P(\text{structure}|\text{data}) \propto P(\text{data}|\text{structure}) \cdot P(\text{structure}) \quad (40)$$

21.2 Dirichlet-Multinomial Framework

For each state σ , we model the emission distribution with a Dirichlet prior:

$$(\theta_0, \theta_1, \dots) \sim \text{Dirichlet}(\alpha, \alpha, \dots) \quad (41)$$

The posterior after observing counts $(c_0, c_1, \dots)$ is:

$$\text{Dirichlet}(\alpha + c_0, \alpha + c_1, \dots) \quad (42)$$

Key Idea

The conjugacy of Dirichlet-Multinomial makes Bayesian updates easy: just add counts to prior pseudocounts!

21.3 Gibbs Sampling for State Assignments

BSI uses Gibbs sampling to explore possible state assignments:

1. Initialize: assign each history to a random state
2. For each history h :
 - Compute probability of h belonging to each state (using Dirichlet-Multinomial)
 - Sample a new assignment proportional to these probabilities
3. Repeat until convergence

21.4 Advantages of BSI

- **Uncertainty quantification:** Get posterior distributions, not point estimates
- **Model selection:** Compare models with different numbers of states
- **Regularization:** Priors prevent overfitting
- **Small data:** Works better with limited observations

21.5 Using emic's BSI

```
from emic import BSI, BSIConfig
from emic.sources import GoldenMean

source = GoldenMean(p=0.5)
data = source.generate(10_000)

config = BSIConfig(
    max_states=5,
    alpha_prior=1.0, # Symmetric Dirichlet
    n_samples=1000,
    burnin=200
)
bsi = BSI(config)
result = bsi.infer(data)

print(f"Inferred {result.machine.num_states} states")
```

22 Causal State Merging (CSM)

Causal State Merging (CSM) takes a bottom-up approach, in contrast to CSSR's top-down splitting. Starting from the finest possible partition—where each unique history of length L defines its own state—CSM progressively merges states whose predictive distributions are statistically indistinguishable.

22.1 Algorithm Intuition

1. **Initialize:** Create one state per unique L -history.
2. **Compute distances:** For each pair of states, measure how different their predictive distributions are.
3. **Merge:** Combine the two closest states (if below threshold).
4. **Repeat:** Continue until no pairs are close enough to merge.

CSM naturally produces a hierarchical clustering (dendrogram) over states, which can be useful for exploring the structure at different granularities.

22.2 Distance Metrics

CSM supports several metrics for comparing predictive distributions:

- **KL divergence:** $D_{\text{KL}}(P\|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$
- **Hellinger:** $H(P, Q) = \frac{1}{\sqrt{2}} \sqrt{\sum_x (\sqrt{P(x)} - \sqrt{Q(x)})^2}$
- **Total variation:** $\text{TV}(P, Q) = \frac{1}{2} \sum_x |P(x) - Q(x)|$
- **Chi-squared:** Useful for hypothesis testing interpretation

22.3 Using emic's CSM

```
from emic import CSM, CSMConfig
from emic.sources import GoldenMean

source = GoldenMean(p=0.5)
data = source.generate(50_000)

config = CSMConfig(
    history_length=5,          # Initial partition granularity
    merge_threshold=0.05,      # Distance threshold for merging
    distance_metric='kl',      # KL divergence (or 'hellinger', 'tv', '
                               chi2')
    min_count=5                # Minimum observations per history
)
csm = CSM(config)
result = csm.infer(data)

print(f"Inferred {result.machine.num_states} states")
```

22.4 Trade-offs

- **Pros:** Avoids over-splitting; produces interpretable hierarchies; robust when data is noisy.
- **Cons:** May under-split if threshold is too aggressive; computational cost grows with number of unique histories; requires choosing history length upfront.

23 Neural State Discovery (NSD)

Neural State Discovery (NSD) uses machine learning techniques to learn state representations. Rather than relying on explicit statistical tests, NSD learns embeddings of histories and clusters them to discover causal states.

23.1 Algorithm Intuition

1. **Embed:** Map each history to a vector representation.
2. **Cluster:** Group similar embeddings using k-means or similar.
3. **Extract:** Build the ε -machine from the discovered clusters.

NSD is particularly useful when:

- The process has complex, high-dimensional structure
- You want to bound the number of states explicitly
- Traditional statistical tests are unreliable due to limited data

23.2 Using emic's NSD

```
from emic import NSD, NSDConfig
from emic.sources import GoldenMean

source = GoldenMean(p=0.5)
```

```

data = source.generate(20_000)

config = NSDConfig(
    max_states=5,           # Upper bound on states
    history_length=10,      # History window for embeddings
    embedding_dim=16,       # Dimensionality of embeddings
    n_iterations=100,       # Clustering iterations
    seed=42                 # For reproducibility
)
nsd = NSD(config)
result = nsd.infer(data)

print(f"Inferred {result.machine.num_states} states")

```

23.3 Trade-offs

- **Pros:** Explicit control over maximum states; can handle complex patterns; reproducible with fixed seed.
- **Cons:** Less interpretable than CSSR/CSM; requires parameter tuning; may not find the minimal model.

Part VI

Worked Examples

24 Example 1: The Biased Coin

The simplest process: independent flips of a biased coin.

- **States:** 1 (all histories are equivalent)
- C_μ : 0 bits (no memory needed)
- h_μ : $-p \log_2 p - (1-p) \log_2 (1-p)$ bits

```

from emic import BiasedCoin, CSSR
from emic.analysis import statistical_complexity, entropy_rate

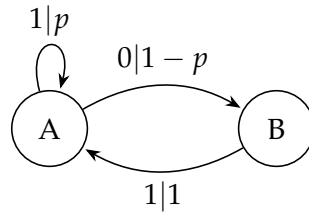
source = BiasedCoin(p=0.7)
data = source.generate(50_000)
machine = CSSR().infer(data).machine

print(f"States: {machine.num_states}") # 1
print(f"Cmu: {statistical_complexity(machine):.3f}") # 0.000
print(f"hmu: {entropy_rate(machine):.3f}") # 0.881

```

25 Example 2: The Golden Mean Process

Binary sequences where 0 must be followed by 1.

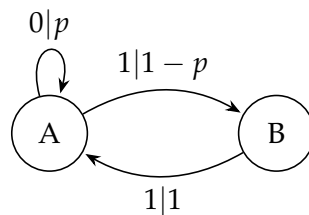


For $p = 0.5$:

- **States:** 2
- **Stationary distribution:** $\pi(A) = 2/3$, $\pi(B) = 1/3$
- C_μ : ≈ 0.918 bits
- h_μ : $= 2/3$ bits

26 Example 3: The Even Process

Binary sequences where 1s come in runs of even length.



Interestingly, for $p = 0.5$, the Even Process has the same C_μ and h_μ as the Golden Mean—they're “complexity twins”!

27 Example 4: Periodic Processes

A period- n process cycles through n states deterministically.

- **States:** n
- C_μ : $\log_2 n$ bits
- h_μ : 0 bits (completely predictable)

```

from emic import Periodic, CSSR
from emic.analysis import statistical_complexity, entropy_rate

for n in [2, 3, 5, 10]:
    source = Periodic(period=n)
    data = source.generate(10_000)
    machine = CSSR().infer(data).machine
    print(f"Period {n}: Cmu={statistical_complexity(machine):.3f}, "
          f"hmu={entropy_rate(machine):.3f}")
  
```

Output:

Period 2: $C_{\mu}=1.000$, $h_{\mu}=0.000$
Period 3: $C_{\mu}=1.585$, $h_{\mu}=0.000$
Period 5: $C_{\mu}=2.322$, $h_{\mu}=0.000$
Period 10: $C_{\mu}=3.322$, $h_{\mu}=0.000$

Part VII

Going Further

28 Connections to Other Frameworks

28.1 Hidden Markov Models

The ε -machine is a special kind of HMM:

- HMMs in general are not unifilar
- The ε -machine is the *minimal unifilar* HMM
- Standard HMM learning (Baum-Welch) doesn't find the ε -machine

28.2 Minimum Description Length

MDL asks: what's the shortest program to describe the data? Computational Mechanics asks: what's the minimal memory for optimal prediction? These are related but distinct.

28.3 Kolmogorov Complexity

Kolmogorov complexity is the length of the shortest program to produce a *single* sequence. It's uncomputable and defined for individuals. C_{μ} is computable and defined for ensembles (distributions).

29 Open Questions

1. **Infinite-state processes:** What happens when $|S| = \infty$?
2. **Continuous processes:** How to extend to continuous alphabets/time?
3. **Quantum processes:** Can quantum mechanics reduce C_{μ} ?
4. **Higher-order structure:** When HMMs fail, what comes next?

30 Further Reading

- **The foundational paper:** Shalizi & Crutchfield [13], "Computational Mechanics: Pattern and Prediction, Structure and Simplicity"
- **Philosophical context:** Crutchfield [5], "The Calculi of Emergence"
- **The CSSR algorithm:** Shalizi & Klinkner [14]
- **Information theory:** Shannon [15], Cover & Thomas [4]
- **Quantum extensions:** Gu et al. [9], Thompson et al. [16]
- **Software:** The emic Python library [7]

References

- [1] W. Ross Ashby. *An Introduction to Cybernetics*. Chapman & Hall, London, 1956.
- [2] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [3] Byron Boots, Sajid M. Siddiqi, and Geoffrey J. Gordon. Closing the learning-planning loop with predictive state representations. In *International Journal of Robotics Research*, volume 30, pages 954–966, 2011.
- [4] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.
- [5] James P. Crutchfield. The calculi of emergence: Computation, dynamics, and induction. *Physica D: Nonlinear Phenomena*, 75(1-3):11–54, 1994.
- [6] James P. Crutchfield and Karl Young. Inferring statistical complexity. *Physical Review Letters*, 63(2):105–108, 1989.
- [7] emic developers. emic: A python library for computational mechanics, 2026.
- [8] Peter Grassberger. Toward a quantitative theory of self-generated complexity. *International Journal of Theoretical Physics*, 25(9):907–938, 1986.
- [9] Mile Gu, Karoline Wiesner, Elisabeth Rieper, and Vlatko Vedral. Quantum mechanics can reduce the complexity of classical models. *Nature Communications*, 3:762, 2012.
- [10] Daniel Hsu, Sham M. Kakade, and Tong Zhang. A spectral algorithm for learning hidden markov models. In *Journal of Computer and System Sciences*, volume 78, pages 1460–1480, 2012.
- [11] Ryan G. James, Christopher J. Ellison, and James P. Crutchfield. Anatomy of a bit: Information in a time series observation. *Chaos*, 21(3):037109, 2011.
- [12] Lawrence R. Rabiner. A tutorial on hidden markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989.
- [13] Cosma Rohilla Shalizi and James P. Crutchfield. Computational mechanics: Pattern and prediction, structure and simplicity. *Journal of Statistical Physics*, 104(3/4):817–879, 2001.
- [14] Cosma Rohilla Shalizi and Kristina Lisa Klinkner. Blind construction of optimal nonlinear recursive predictors for discrete sequences. pages 504–511, 2004.
- [15] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [16] Jayne Thompson, Andrew J. P. Garner, John R. Mahoney, James P. Crutchfield, Vlatko Vedral, and Mile Gu. Causal asymmetry in a quantum world. *Physical Review X*, 8(3):031013, 2018.